

Seminar Report on Express: Lowering the Cost of Metadata-hiding Communication with Cryptographic Privacy

Carina Fehr (*remember to remove ID for the first draft and peer review!*)

October 12, 2025

Abstract

This report studies the reproducibility and performance of Express, a system for metadata-hiding communication. Using two servers, express ensures cryptographic security against at most one malicious server and an arbitrary number of malicious clients. The paper presents new protocols that offer low communication overhead, regardless of the number of users, and improved computation costs using symmetric key cryptography. Compared to Pung and Riposte, two previous cryptographically secure systems, Express achieves increased message throughput, while maintaining strong privacy guarantees. This makes Express an ideal applications for communication with whistleblowers.

1 Introduction

In today's digital landscape, secure communication has become increasingly crucial, with growing capabilities of surveillance and metadata analysis. Secure messaging apps like Signal or Threema and Transport Layer Security (TLS) encrypt the content of sent messages, but they do not protect metadata. It is possible to observe who sent and received messages, when and how often they communicated and how much data they exchanged. These information are as sensitive as the message itself and should therefore not be leaked. It is challenging to keep all data hidden, as most services for communication over the internet rely on some of the metadata to proceed the messages. An option for metadata hiding would be using a TOR browser or SecureDrop, but they are vulnerable to attacks by adversaries that control many network points. Other existing systems to hide metadata either have a high communication cost to be used in reality, or they have a low computation cost but their security gets worse with every round of communication, also making them impractical.

Express offers a new communication option that keeps metadata private while ensuring feasible costs and calculation times. While two servers are required, Express still runs with one malicious server (Eskandarian et al., 2021). Express' target audience are whistleblowers. To receive messages, a journalist or any other person that should receive information registers a mailbox with the two servers. The server assigns a 128-bit virtual address to each mailbox and maintains a page table in which it stores the corresponding physical address. The client can transmit this address to the whistleblower. Now the client can upload a message into the mailbox. No one except the mailbox owner will know that a message has been sent into this mailbox. Mailbox owners have to check their mailboxes regularly. Express does not hide when a client read from a mailbox, therefore a pattern in the interval of checking could reveal something about activity of this mailbox. Additionally, Express must implement a possibility for cover traffic. This provides whistleblowers with plausible deniability and ensures that they cannot be accused of giving information simply for contacting an Express Server.

There are different types of attacks that Express must be resilient against. All clients can read from all mailboxes, so Express must re-encrypt the mailboxes differently each time a read happens, providing an encryption only the owner can decrypt. All others can neither read the content nor learn anything about which client is writing, even when reading the content of all mailboxes between every pair of writes. Another possible threat would be a malicious client targeting all mailboxes. It could overwrite all mailboxes with one message. Because the mailboxes can only hold a maximum of one message, all messages sent before this will be lost due to this denial-of-server attack. The complete overview is divided between the two servers, which means that the servers alone cannot recognise that this is not a valid message. A new auditing protocol presented in the paper is used to prevent such attacks. However, a malicious client could

also attack just one mailbox, in which case the message would be completely correct and the server would have no way of detecting and preventing the attack. Virtual addresses are used to prevent the original messages in the mailbox from being overwritten. The address range is so large that the probability of hitting a specific mailbox is negligible.

2 Background

This section explain two key concepts that form the foundation of Express: cryptographic privacy and distributed point functions, also called DPFS. In the following section, we will show how they can be used to achieve efficient metadata-hiding communication.

2.1 Cryptographic privacy

Cryptographic privacy is a mechanism that guarantees no information about communication metadata leaks to an adversary. Even after observing many rounds of communication, a global adversary who can observe all network traffic cannot determine which clients are communicating with each other or which mailbox a message was written to.

An alternative mechanism is differential privacy. After several rounds of communication, systems based on differential privacy such as Vuvuzela or Stadium, will loose security. Consistent observation will allow the adversary to learn something about communication patterns. Therefore, although cryptographic privacy is slower and involves higher computational cost, it is the preferred approach for frequently used applications where long-term security is essential.

2.2 Distributed point function

A DPF (Boyle et al., 2023) is a cryptographic algorithm. It splits a function f into function shares that individually hide f . The point function $f_{a,b}(x)$ outputs b at the point $x = a$ and 0 everywhere else. The two or more parts of the function individually reveal nothing about the function.

Express splits the function into two parts, one for each of the servers. Because no server alone can reconstruct the full function, even a malicious or compromised server cannot tell anything about the function and learns nothing about which mailbox a client wrote into.

3 Private Communication in Express

In this section, we will explain how an Express client writes messages into mailboxes and how the servers store them (see Figure 1). To send a message, the client creates a zero vector with the length of all mailboxes. It inserts its message into this vector at the relevant position in its target mailbox and sends a portion of this vector to both servers. The servers add this vector to their current status.

To read a message, the client sends the physical and virtual address of its mailbox to servers A and B. The servers check their page table to see if this is a correct combination of physical and virtual addresses. They then send the contents of the relevant mailbox D_{Ap} and D_{Bp} as well as the corresponding keys k_{Ap} and k_{Bp} used for encryption. The servers then empty this mailbox by setting the values of D_{Ap} and D_{Bp} to a new encryption of zero with the keys K_{Ap} and K_{Bp} . The owner of the mailbox can now use the key to decrypt the contents received from both servers. They add these parts of the messages m_{Ap} and m_{Bp} and receive their secret message m .

3.1 Express Servers

Secret sharing with DPFs. Servers A and B together store the entire contents of all mailboxes. Both hold a share of the secret messages, which is implemented using DPFs. If the express servers hold n mailboxes and each of them contains an element of the finite field $\mathbb{F}$, the totality of all mailboxes can be represented as a vector: $D \in \mathbb{F}^n$. D can therefore be represented as the sum of the two components $D_A + D_B = D$, where the vector belonging to server A corresponds to $D_A \in \mathbb{F}^n$ and server B's vector corresponds to $D_B \in \mathbb{F}^n$. When a clients sends a new message, they send both servers one share of a vector w containing this message (further details in "Express Clients"). When the servers receive the message w_a and w_b , they can both add

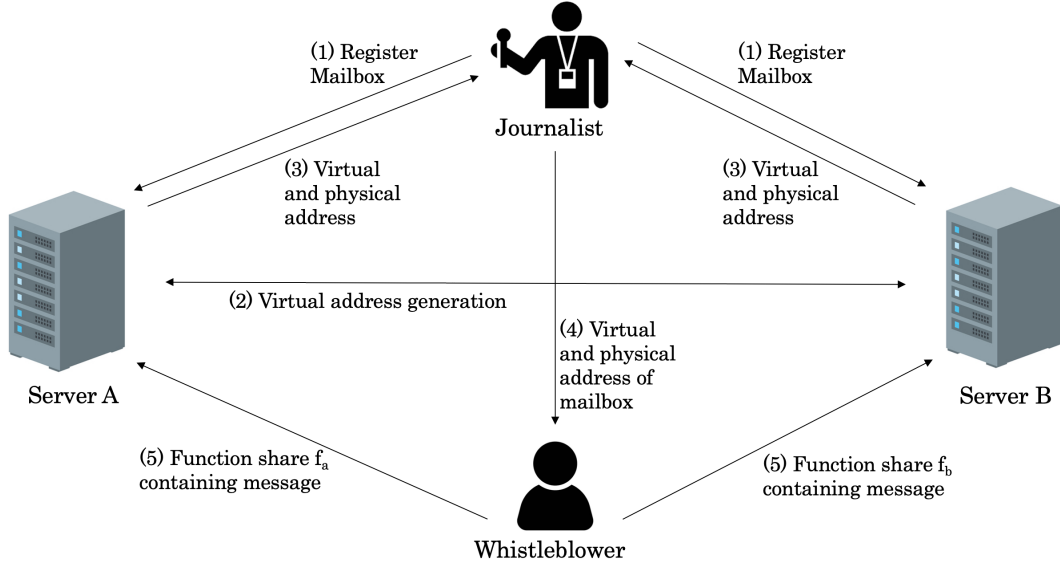


Figure 1: Communication between servers and clients. The figure shows the numbered steps from registering a mailbox to sending a message. It does not include the retrieval of a message by a client.

this vector to their current status: $D_A \leftarrow D_A + w_A \in \mathbb{F}^n$ and equally $D_B \leftarrow D_B + w_B \in \mathbb{F}^n$. This ensures that the received message is placed in the correct mailbox and all other mailboxes remain unchanged.

By using DPFs instead of sending such large vectors the servers receive only a function share. They can evaluate this at any point, i.e. for each mailbox. This allows them to recover the vectors w_a and w_b , which they can add to D_A and D_B as before. Chapter xxx explains in more detail how the functions are created.

Distribution of trust. To prevent one server from containing all the data, thus representing a single point of failure and not providing adequate security for metadata hiding, multiple servers must be used. One way to achieve this distribution of trust would be to use a large number of servers. This offers the advantage that even if more than one server is malicious, the system can still continue to run.

Express uses only two servers, which in the paper are the ones from Wall Street Journal and the Washington Post. These two must therefore meet stricter security requirements. One difficulty in implementing Express could be censorship. If access to one of the two servers is blocked, the entire application will no longer function. Express is therefore less suitable for countries where news websites, i.e. servers on which Express runs, are often restricted.

Malicious server Another danger is posed by a malicious server. A denial of service attack can originate not only from a client, but also from a server. The server can overwrite the contents of all mailboxes with meaningless content, causing the original content to be lost. However, since the client will realise that the contents of their mailbox are garbage, they can recognise that the system can no longer be trusted.

3.2 Express Clients

The messages sent by clients should either all be the same size or consist of a predefined set of sizes. If an attacker can extract information from the message size, the system loses security and cover traffic no longer provides sufficient plausible deniability. If a client wants to write a message $m \in \mathbb{F}$ to a mailbox, it can insert it into a vector of the length of all mailboxes by multiplying it with the i -th standard-basis vector e_i . This vector's value should be one at the location of the targeted mailbox and zero everywhere else. Mathematically, this can be formulated as follows: $m \cdot e_i \in \mathbb{F}^n$. This vector must then be split so that to each server one share can be sent. When added together, they form the vector with the message, i.e. $w_A + w_B = m \cdot e_i$. No relevant information can be obtained from one vector share alone.

However, the costs of these calculations are high. Each message sent by the client consists of a vector with a length equal to the number of mailboxes in the system. This vector therefore quickly becomes very large, and the fact that both servers receive a vector further increases communication costs. A better alternative is to use DPFs, which allows clients to send less data by compressing the vectors w_a and w_b . As explained in 2.2, it is possible to split a function into two parts that, on their own, do not reveal anything about

the function. We call the functions for the two Servers A and B f_A and f_B . When the functions are added together, they result in the function f . f maps indices to function values, i.e. mailboxes to the messages they contain. The function can therefore be evaluated at any point to determine the contents of the corresponding mailbox. This point function can also be written as follows: $f_{i^*,m} : [N] \rightarrow \mathbb{F}$. i^* is the only point at which this function does not yield zero, but rather m . Clients are able to generate the shares f_A and f_B . Added at any point $i \in [N]$, they result in the DPF: $f_A(i) + f_B(i) = f_{i^*,m}$. Individually, these do not tell an attacker anything about i^* , i.e. the mailbox being written to, or about m , i.e. the message being transmitted. The use of DPFs has reduced the size of sent messages to such an extent that, with a security parameter λ , each part of the function now has a length of $O(\lambda \log N + |m|)$ bits.

4 Protecting Privacy and Integrity

In this section we first show how Express ensures privacy when all clients behave legitimately, and then when at least one is malicious. Additionally, we will show how to provide security when both a client and one of the servers are malicious. At the end, we will show how Express can be used as a tool for communication between whistleblowers and journalists.

4.1 Privacy with honest clients

The previous chapter 3 explained how we can write a message secretly to the server. However, an adversary could now check all network traffic. If they compare the contents of all mailboxes after each write, they will not be able to read the message because it is encrypted, but they can find out which client wrote to which mailbox.

To ensure that the metadata is completely hidden, we also need to control who reads the mailboxes. This could be prevented by only allowing reading once all clients have written, i.e. by running the system in batched rounds. Pung and Riposte are based on this concept. However, this severely limits actual usability. Express has therefore proposed a method that does not require synchronised rounds. To achieve this, we must ensure that an attacker cannot view the messages or identify which mailbox has been modified after a message has been sent. The only person entitled to this information is the owner of the mailbox. If a mechanism is found that meets these two requirements, a network adversary can still determine when something was written, but will not learn anything about the modified mailbox.

For this, Express uses a lightweight cryptographic algorithm based on symmetric cryptography. The servers do not need to authenticate the clients when they submit a read request. Before changing the contents of a mailbox, the servers must decrypt it and re-encrypt after the modifications. Both servers have their keys k_A and k_B , which a client uploads when creating a mailbox. The re-encryptions after the write are randomized. This means that an attacker reading the encrypted contents cannot find out which mailbox a new message has been stored in unless they have both keys.

A counter is used for encryption. So when re-encryption is required, the previous value is subtracted and the new value added, which is a commutative operation. It is not necessary to re-encrypt after every write, only if a read has occurred between writes. Otherwise, no one, including an attacker, has obtained any information. This increases the efficiency of encryption. Additionally, it is not absolutely necessary for both servers to perform the re-encryption. If one of the servers has been corrupted and no longer re-encrypts, the system remains secure.

In section 4.2.2 we will explain that auditing does not protect against attacks on a single mailbox and how we prevent these attacks instead.

4.2 Privacy with malicious clients

The techniques presented in the previous chapter only provide security as long as all clients behave honestly. Even a single malicious client can attack and damage both servers. Nothing prevents a client from creating a vector that writes to multiple mailboxes simultaneously. To do this, it creates a DPF whose added vectors w_A and w_B , calculated from the functions f_A and f_B , are not equal to zero on multiple indices. The mailboxes at Express can only hold one message at a time. If a new one is written, the old one can no longer be read. Thus, one message from a malicious client is sufficient to overwrite multiple or even all mailboxes and thereby delete the original content. Express has introduced an auditing protocol to prevent this scenario.

This protocol ensures that each write updates only one mailbox. Each vector $w = (w_a + w_b) \in \mathbb{F}^n$ to be inserted must therefore have zero as an element at every position except one. If the vector does not meet this requirement, it can be discarded, meaning that the write request is rejected by the server and nothing is written to any mailbox.

Another danger posed by a malicious client is a targeted attack on a specific mailbox. The client, which can no longer write to multiple mailboxes due to the auditing protocol, still has the ability to write a message to a specific mailbox. We will explain these two concepts in more detail in the following two subchapters.

4.2.1 Auditing Protocol

The auditing protocol must meet three conditions, which are explained in more detail in this chapter. The first property is completeness: if all parties are honest, the auditing protocol always accepts. In addition, soundness must be fulfilled. If a malicious client makes a write request that is not valid, the two honest servers must reject it. And finally, zero-knowledge must also be fulfilled: a malicious server cannot learn anything from a write request from an honest client, except that the request is valid.

Express has developed a new auditing protocol that differs from Riposte's previous one in terms of the number of servers required, as well as the computation times and costs. Riposte requires three servers, only one of which may be malicious. The time required for a write in Riposte is linear, which leads to high costs for clients. The auditing protocol developed in Express has a constant runtime, making it much more suitable for real-world applications. In section 5, we will compare the performance of these two systems in more detail.

The task of the auditing protocol is to check the Hamming weight of the received vector w , i.e. the number of non-zero positions. Only if this is one, the message is valid. An n -variant polynomial is used that is based on shared randomness between the two servers and does not require any interaction between them. If there is exactly one non-zero w_i and this corresponds to the message $m = w_i$, then the polynomial will most likely result in zero for random choices of $r_1, \dots, r_n$: $f(r_1, \dots, r_n) = (\sum_{i \in [n]} w_i r_i)^2 - m(\sum_{i \in [n]} w_i r_i^2)$. This formula checks whether the square of the random linear combination and the sum of the terms of the linear combination are equal to the square. This approach is only secure against semi-honest servers, i.e., servers that follow the protocol but then attempt to learn information they should not know. Malicious clients, on the other hand, could deviate from the protocol and thereby obtain information about which mailbox the client wants to write to by finding the entry w_i . The malicious server can modify the received vector and write content to a specific index i . If the audit fails and the combined vector $w_A + w_B$ is rejected due to its hamming weight being not equal to one, the server knows that the client from which the vector originated does not want to write to mailbox i . If, on the other hand, the message is accepted, the server can be sure that the write request it got would modify the index i .

To protect against malicious servers, Express performs its auditing protocol within the secret-shared non-interactive proof (SNIP) system (Corrigan-Gibbs and Boneh, 2017). The exact functionality of SNIP proofs is treated as a black box here and explained in more detail in the appendix of express (Eskandarian et al., 2021). Each client sends an arithmetic circuit "verify()" in addition to its share of w . The servers that only have w_a / w_b can compare with each other whether $\text{verify}(w)=1$. To do this, they need to be able to rely on shared randomness. During setup, servers A and B agree on a shared secret. Using a pseudorandom generator, they can now create a new, shared random seed r for each execution of the protocol. The servers first send r to the clients. Then they calculate the message shares m_a and m_b by summing up elements in their received vectors w_a and w_b , because only one element is not zero. The servers generate several random numbers using their random seed and insert them into a vector $r = (r_1, \dots, r_n) \in \mathbb{F}^n$. They then square each element of this vector to get $R = (r_1^2, \dots, r_n^2) \in \mathbb{F}^n$. By combining these values, the servers can create their check values, which represent the inner product between the vectors: $c_A \leftarrow \langle w_A, r \rangle \in \mathbb{F}$, $c_B \leftarrow \langle w_B, r \rangle \in \mathbb{F}$, $C_A \leftarrow \langle w_A, R \rangle \in \mathbb{F}$, $C_B \leftarrow \langle w_B, R \rangle \in \mathbb{F}$.

The client must then prepare its proof input to convince the server that its input is legitimate. From the server it has received r and the client also knows the index i^* at which it wants to write. It can now derive r^* and r^{2*} , which is r at the position of i^* , i.e., a secret random number associated with the mailbox being written into. The client now calculates the check values of its two vector shares by multiplication: $c^* = r^*(w_{Ai^*} + w_{Bi^*})$ and $C^* = r^{2*}(w_{Ai^*} + w_{Bi^*})$. Because the client knows the index i^* of its message, it only has to perform these calculations for this entry and its time complexity is $O(1)$. The servers, on the other hand, have to perform the calculations for all elements, which means they need $O(n)$.

The client must now prove that $c^2 - m \cdot C = 0$ where $c \leftarrow c_a + c_b$ and $C \leftarrow C_A + C_B$. To do this, it

prepares a SNIP proof $\pi = (\pi_A, \pi_B)$. It sends π_A to server A and π_B to server B. The servers can now verify the proof together through communication. The soundness property of the SNIP proof ensures that the servers accept a proof only if the corresponding statement is valid. At the same time, the zero-knowledge property guarantees that, as long as at least one server behaves honestly, the verification process reveals no additional information about the client's write request, even if the other server acts maliciously.

SNIP proof's computation and communication time are linear in the number of multiplications between secret values. The formula from the auditing protocol only requires two multiplications, two squarings and one multiplication by m , the SNIP proof runs in constant time. As the performance bottleneck on the server is the evaluation of DPFs, improvements in auditing would not result in significant progress of the overall performance.

4.2.2 Targeted attacks

An adversary can write random data to a specific mailbox. The recipient can easily determine that the contents of their mailbox contain no meaningful content and that the original contents must have been deleted. However, since these messages are sent in a formally correct vector, it is not possible to intercept them with a protocol.

Express uses virtual addresses for its mailboxes to solve this problem. Each mailbox is assigned a 128-bit virtual address. Clients that want to write to a mailbox must know this address. While the physical address space holds about 2^{20} entries, the address space has a size of 2^{128} with 128-bit addresses. This is equal to $3.4 \cdot 10^{38}$, meaning there is such an enormous amount of virtual addresses that most of them are never used and an attacker is never able to guess a victim's mailbox.

Internally, this is implemented by page tables. Express maintains an array of n physical addresses and an array of 2^λ virtual addresses, where λ is a safety parameter. The servers jointly maintain a page table in which they map the physical addresses to the virtual ones. These tables are stored without encryption on the servers, as a malicious client cannot obtain any useful information from them. When a client registers a new mailbox, the servers assign it a random virtual address and the corresponding physical address and return these two to the client. Either one server chooses the virtual address itself and communicates it to the other, or both servers determine one, then they use shared randomness to arrive at the same address. The servers reserve memory for a new physical address and add this and the created virtual address to the page table.

A client that wants to send a message creates a DPF with the shares f_A and $f_B : 2^\lambda \rightarrow \mathbb{F}$ as if it were writing to the virtual address space. When creating the message, the client must know both the physical and virtual addresses of its destination mailbox, otherwise its DPF cannot be checked by the auditing protocol. Using its page table, the server can now evaluate only the function at the locations where a mailbox actually exists. This prevents the servers from having to perform an exponential number of calculations, which would result in excessively long computation times.

4.3 Express for Whistleblowing

As explained in the previous chapters, Express offers a secure communication option. Outside of the data transfer itself, there are two more safety issues that should be considered. First, we need plausible deniability, second, we need a way to agree on a Mailbox address.

4.3.1 Cover traffic

If whistleblowers were the only users of Express, it would be obvious that anyone who sent a message over Express had passed on information. This means that many users that are not involved in whistleblowing must also send messages, creating cover traffic. To achieve this, a JavaScript program can be integrated into a website with high traffic, such as a news site. Every time a user visits this website, a dummy request is sent to the Express servers. The servers generate a key that is not known to any client. This means that these messages cannot be read, but they also cannot be distinguished from those of the whistleblowers.

The cost and computation times are very low, so that these do not pose an issue for integrating Express scripts on the website. The loading times of the New York Times, Washington Post and Wall Street Journal were analysed. They take between 2 and 5.5 seconds to load completely. The Express Client needs 51 ms in JavaScript to send 1 KB of data. This means that the calculation times of the Express Client in the browser

correspond to less than 3% of the current loading time. The JavaScript code would only increase the size of these pages by less than 1.5%. The JavaScript can therefore be executed in the background without affecting the user.

News websites like New York Times benefit from high cover traffic because whistleblowers feel safer and are more likely to pass on information. This makes such sites particularly suitable for integrating the Express code. Express does not specify how much cover traffic is required exactly, but it should be enough to ensure that real messages remain indistinguishable from the generated ones.

4.3.2 Dialing

To select a mailbox address, the journalist and the whistleblower can meet in person. However, this is often not possible. In this case, a dialling protocol can be used. For example, the Riposte system could be used as a basis and made faster and cheaper with the improvements identified in Express. Journalists post their public keys for Riposte online, then whistleblowers can send them a mailbox address. The Journalist can download their messages and register the mailboxes.

5 Related work

In this section we will present Pung, Riposte and Addra, three cryptographic metadata-hiding communication systems working with the concept of mailboxes. Additionally we present Securedrop, a system developed for whistleblowing that has been in use since 2013. SecureDrop, Pung and Riposte were developed before Express and were used as sources in the paper on Express. Addra, on the other hand, was developed later and uses Express as a source.

SecureDrop handles the same problem as Express, but takes a different approach to privacy and does not offer cryptographic security. Pung addresses the same problem as Express, focusing on a solution for whistleblowers, but uses more computationally intensive public key cryptography. Riposte uses similar methods to Express, but is not suitable for whistleblowers because messages are not secret. Addra extends the goal of metadata-hiding even further, presenting a solution for making secure and private voice calls (Ahmad et al., 2021). Then we will compare their security and performance to Express.

5.1 SecureDrop

SecureDrop (Swartz, 2013) is an open-source whistleblowing platform (Berret, 2016). It is used by news organizations to enable anonymous, secure exchange of documents and messages between sources and journalists. Maintained by the Freedom of the Press Foundation, it is deployed by numerous media organizations worldwide, including The New York Times and The Guardian. SecureDrop was originally developed by Aaron Swartz, an activist against internet censorship, and Kevin Poulsen, who was earlier pursued by the FBI and spent five years in prison for computer fraud. SecureDrop aims at protecting the identity of whistleblowers and the confidentiality of leaked information. Sources submit documents and messages through the Tor network, which hides both the network location and identity of sources from the news organization and potential adversaries. The system stores encrypted submissions on physically secured, disconnected from the wider organization network's disconnected servers. Journalists access these servers through separate, hardened machines that are kept offline to prevent data leaks. While SecureDrop provides strong content confidentiality and reasonable anonymity at the network level, it does not achieve cryptographic metadata privacy.

5.2 Pung

Pung (Angel and Setty, 2016) is designed to provide privacy guarantees of both message content and metadata. Its focus lies on protection against powerful global adversaries who may control the entire network infrastructure. This could be mass surveillance or state-sponsored attacks. In Pung, clients deposit and retrieve messages through an untrusted key-value stores. Only one server is required. Pung provides privacy even when the server is malicious.

Pung's main goal is to hide the communication patterns of clients who already know each other's identities. It allows two or more users to exchange messages over a public network without anyone else learning that they communicated. To prevent network surveillance, Pung operates in rounds. In each round, a client

can only send and receive a limited number of messages. If this limit is exceeded, users must wait for the next round. Message retrieval is based on private information retrieval (PIR). As a result, clients have to download large amounts of data simultaneously to ensure anonymity. Pung is therefore particularly suitable for applications where this is the case, such as emails or group chats. Otherwise, the cost of network traffic is relatively high.

While significantly reducing the costs compared to prior approaches, Pung's network costs still remain high. However, newly introduced techniques have significantly reduced the cost of cryptographic calculations.

5.3 Riposte

Riposte protects against network surveillance and malicious clients (Corrigan-Gibbs et al., 2015). Like Express, the main target group is whistleblowers who want to leak information anonymously. Riposte users can post messages anonymously on a bulletin board, which acts as a public broadcast channel. Clients have a fixed time window in which they can write their messages. These are collected on the server and published when the time window has expired. Now everyone can read all the messages written previously. The content is therefore not secret, but the author remains anonymous. If the messages were published after each write, everyone could see that the person who was last connected to the server created that message.

Riposte can be used in two ways. Either with three servers, only one of which may be malicious, or with a variable number, at least one of which must be honest. The variant with three servers, at least two of which are honest, uses a less expensive protocol. It is therefore preferable when millions of clients use Riposte, and it is also the one compared with Express in the paper.

Using DPFs and a private information retrieval (PIR), Riposte minimizes the number of bits each client must upload to each server.

Riposte's key innovation was minimizing the amount of data each client uploads to the servers, making large-scale anonymous writing more practical than earlier systems. However, it relies on public-key cryptography and synchronized communication rounds, which still lead to computational and communication overhead.

5.4 Addra

Addra is dedicated to hiding metadata during voice calls (Ahmad et al., 2021). This data, such as call duration, time, parties involved, can be very interesting for government surveillance programmes and is therefore highly sensitive. Addra is the first system to hide metadata from voice communications over a completely untrusted infrastructure. Previous systems with the same goal were not enough scalable, with a maximum of a few dozen users, or they required trusted partners, which posed a security risk.

Callers receive a mailbox when they initiate a call, regardless of who it is going to. They place their message in it, preventing an enemy from finding out who the call is going to. Using the PIR protocol, the callee can retrieve this message from the mailbox. Addra extends Pung's protocol using public and private keys to establish secure communication channels. Addra uses Pung's PIR protocol with private and public key encryption. However, Addra has expanded it to FastPIR. This reduces calculation times on the server side. In addition, PIR calculations are parallelised, as low latency is essential for calls.

Addra has demonstrated that metadata privacy is also important for more demanding areas than sending messages and can be implemented using similar methods.

5.5 Comparison

All five application focus on combining scalability with strong privacy guarantees against network surveillance and malicious servers. A significant limitation of Pung and Riposte compared to Express is that both work with batched rounds, meaning each client must have written their message before anyone can read it. This simplifies anonymity guarantees, but restricts usability, since communication can only proceed once all clients in a round have acted. Batch processing introduces high latency and makes real-time interaction impractical.

Express eliminates this limitation through an asynchronous architecture that allows clients to write and read messages independently. This not only improves responsiveness but also reduces the coordination overhead that previously limited scalability. Express therefore offers comparable cryptographic privacy with a better user experience. Riposte focuses on anonymous public posting, functioning as a broadcast channel

rather than private messaging. Pung and Addra on the other hand enable private message exchange between known parties over an untrusted infrastructure. However, both systems achieve this by accepting higher computational and network costs, which limits their scalability to smaller user groups.

SecureDrop works most differently compared to the other four applications. It is vulnerable to traffic analysis if a large amount of the network is observed. Because metadata is not cryptographically hidden, an adversary could still detect that communication occurred between a whistleblower and a journalist, even if they cannot read the message content. Furthermore, maintaining SecureDrop relies on a trusted infrastructure within the media organization, including physical security for the servers and trained operational staff.

In times when technical possibilities are constantly expanding and nation-states have a high interest in metadata, relying on trusted servers is risky. States can use their enormous political and financial resources to attack these servers. That is why Pung and Addra, which continue to operate even without a trusted infrastructure, offer the strongest security. Riposte and Express take a different approach by distributing trust across two or more servers. Compared to Riposte, Express strengthens its security guarantees through an auditing mechanism that detects malformed client requests and prevents data corruption without revealing metadata. This protection against malicious clients makes Express more robust in practical deployments while still maintaining a realistic trust model. Therefore, Express provides a balanced compromise between strong cryptographic privacy and operational practicality, making it more suitable for large-scale or real-world use.

5.5.1 Performance

Significant events prior to the research of Express have led to Express testing larger messages than previous systems. In 2019, a whistleblower submitted a report about President Trump and his interactions with the leader of Ukraine (Anonymous, 2019). The file had a size of 25.3 KB and showed how important it is to offer whistleblowers the option of sending long messages.

Overall Express reduces the costs of maintaining a service for whistleblowers by $6\times$ compared to previous work with the same objective.

In Express, the communication cost were compared to that of Riposte and Pung. To do this, Eskandarian, Corrigan-Gibbs, Zaharia and Boneh set up three Google Cloud instances, each with 54 GB of RAM. All three run on the same data centre to minimise latency. These were used to evaluate Express, Pung and Riposte 1.

Table 1: Communication cost for 1 million mailboxes

	Express	Pung	Riposte
Servers needed	3	1	2
Client side	x	$101\times x$	$4631\times x$
Server site	y	$195\times x$	$7161\times y$

Since Express and Riposte have multiple servers, these two applications have an auditing protocol. For a fixed security parameter λ , Express' auditing protocol requires $O(\lambda)$ communication between parties. For the client side communication, time is constant $O(1)$ with less than 5 microseconds. Riposte on the other hand required $\Omega(\lambda\sqrt{n})$ communication and $\Omega(\sqrt{n})$ client computation. In addition, Riposte requires three servers instead of two like Express. This results in the new auditing scheme reducing the costs for the client by $8\times$ for an installation with one million registered mailboxes.

Table 2: Express vs Riposte costs

	Express	Riposte
Auditing protocol	x	$8\times x$
Throughput	y	$1.4-6.3\times y$
Process 1 million messages	z	$5.9\times z$

The table 2 shows that express achieved improvements in runtime compared to riposte across all operations. It is noticeable that Express not only has a faster auditing protocol, but also has a higher throughput. In the same time that Express can process 32 KB, Riposte only manages to process approximately one KB,

measured with approximately 50,000 mailboxes. Pung is also slower than Express. The difference is particularly noticeable with small messages. With messages of 10KB, Pung is still about 1.3-2.6 times slower than Express, and with 32KB they are about the same speed.

5.5.2 Cost analysis

In their cost analysis, the authors limited themselves to the cloud infrastructure, so engineers are not included. The biggest cost factor is server operation. Public Google Cloud Platform pricing information can be used to determine the cost in dollars of sending one million messages. The calculated cost consists of the cost of hosting the servers for the length of time required to process these messages, additioned to the data passed between the servers and back to the clients. The cost of Express for this million messages is 5.9 times less than Riposte. Pung is again significantly more expensive than Riposte. Sending one million messages costs over \$1,000 with Pung, although it only operates one server. The main costs are for data egress. Express requires significantly less money for the same number of messages, namely only \$0.05 and Riposte \$2.41. Since express only has to maintain two servers and not three like Riposte, it runs relatively cheaper per hour.

6 Conclusion

We have shown that Express made progress in performance and reproducibility of privacy enhancing technologies. Its auditing and virtual addressing achieve efficient, scalable metadata privacy suitable for whistleblowing, journalism, or any communication system where metadata leakage is a risk. We compared Express with three other systems, Pung, Riposte and Addra, that protect metadata from attackers. By comparing the calculation costs and computation times of the different applications, we found that Express is more cost-effective and time-efficient without sacrificing privacy guarantees.

7 Use of generative digital tools

The use of AI tools must always be declared completely and transparently. For examples of how to do this, please see the DMI guidance: https://dmi.unibas.ch/fileadmin/user_upload/dmi/Studium/Computer_Science/Diverses/Wegleitungen/Dokumente_2023/Guidance_on_AI_in_teaching_in_Computer_Science.pdf.

References

- Ahmad, I., Yang, Y., Agrawal, D., Abbadi, A. E., and Gupta, T. (2021). Addra: Metadata-private voice communication over fully untrusted infrastructure. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 313–329. USENIX Association.
- Angel, S. and Setty, S. (2016). Unobservable communication over fully untrusted infrastructure. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 551–569, Savannah, GA. USENIX Association.
- Anonymous (2019). Whistleblower Complaint | DocumentCloud — [embed.documentcloud.org. https://embed.documentcloud.org/documents/6430351-Whistleblower-Complaint/?mode=document&embed=1](https://embed.documentcloud.org/documents/6430351-Whistleblower-Complaint/?mode=document&embed=1). [Accessed 09-10-2025].
- Berret, C. (2016). Guide to securedrop.
- Boyle, E., Gilboa, N., Ishai, Y., and Kolobov, V. I. (2023). Information-theoretic distributed point functions. Cryptology ePrint Archive, Paper 2023/028.
- Corrigan-Gibbs, H. and Boneh, D. (2017). Prio: Private, robust, and scalable computation of aggregate statistics. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 259–282, Boston, MA. USENIX Association.

- Corrigan-Gibbs, H., Boneh, D., and Mazieres, D. (2015). Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE Symposium on Security and Privacy*, page 321–338. IEEE.
- Eskandarian, S., Corrigan-Gibbs, H., Zaharia, M., and Boneh, D. (2021). Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1775–1792. USENIX Association.
- Swartz, A. (2013). Share and accept documents securely — securedrop.org. <https://securedrop.org>. [Accessed 11-10-2025].



University
of Basel

Faculty of Science



Declaration on Scientific Integrity

(including a Declaration on Plagiarism and Fraud)

Translation from German original

Title of Thesis: _____

Name Assessor: _____

Name Student: _____

Matriculation No.: _____

I attest with my signature that I have written this work independently and without outside help. I also attest that the information concerning the sources used in this work is true and complete in every respect. All sources that have been quoted or paraphrased have been marked accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using the appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or expulsion from the study program.

Place, Date: _____ Student: _____

Will this work, or parts of it, be published?

No

Yes. With my signature I confirm that I agree to a publication of the work (print/digital) in the library, on the research database of the University of Basel and/or on the document server of the department. Likewise, I agree to the bibliographic reference in the catalog SLSP (Swiss Library Service Platform). (cross out as applicable)

Publication as of: _____

Place, Date: _____ Student: _____

Place, Date: _____ Assessor: _____

Please enclose a completed and signed copy of this declaration in your Bachelor's or Master's thesis.

September 2023